

Integrating Genie One MCP Server into Databricks Apps

Overview

The **Genie One MCP server** (`system.ai.genie_one_mcp`) is a Databricks-provided MCP Service that exposes Genie One as a conversational tool over the Model Context Protocol (MCP). It lets you send natural-language data questions from your custom Databricks App and receive answers grounded in your enterprise data — **without creating a dedicated Genie Agent**.

Genie One automatically routes questions to the most appropriate Genie Agent, searches dashboards, queries, and metric views, and resolves business terms through the Genie Ontology.

Prerequisites

1. **Premium tier workspace** — Databricks Apps require a Premium workspace.
 2. **Python 3.12+** in your development environment.
 3. **OAuth authentication** configured for your workspace.
 4. **Unity Catalog permissions** — Account users hold EXECUTE on `system.ai` by default, so no extra grant is usually required.
 5. **Serverless compute** must be enabled in your workspace.
-

Step 1: Set Up Your Local Environment

Authenticate to your workspace using OAuth:

```
databricks auth login --host https://<your-workspace-hostname>
```

When prompted, enter a profile name (default is DEFAULT).

Install the required Python dependencies:

```
pip install -U "mcp>=1.9" "databricks-sdk[openai]" "mlflow>=3.1.0" "databricks-agents>=1.0.0" "databricks-mcp"
```

For agent frameworks, also install:

- **OpenAI Agents SDK:** `pip install openai-agents databricks-openai`
 - **LangGraph:** `pip install databricks-langchain langgraph`
-

Step 2: Identify the Genie One MCP Service URL

The Genie One MCP server is available at:

```
https://<workspace-hostname>/ai-gateway/mcp-services/system.ai.genie_one_mcp
```

You can also view your MCP Services and their URLs in the workspace UI: **Unity Gateway** → **MCPs**

Note: The previous Beta endpoint (<https://<workspace-hostname>/api/2.0/mcp/genie>) is deprecated and will be sunset on **October 31, 2026**. Use the new `system.ai.genie_one_mcp` endpoint.

Step 3: Connect and List Available Tools

Create a `DatabricksMCPClient` and verify the available tools:

```
from databricks_mcp import DatabricksMCPClient
from databricks.sdk import WorkspaceClient

workspace_client = WorkspaceClient(profile="DEFAULT")
host = workspace_client.config.host

mcp_server_url = f"{host}/ai-gateway/mcp-services/system.ai.genie_one_mcp"
mcp_client = DatabricksMCPClient(
    server_url=mcp_server_url,
    workspace_client=workspace_client
)

tools = mcp_client.list_tools()
print(f"Available tools: {[t.name for t in tools]}")
```

The Genie One MCP server exposes these tools:

Tool	Description
genie_ask	Ask Genie a natural-language data question. Returns <code>conversation_id</code> , <code>response_id</code> , and <code>status</code> . Pass a <code>conversation_id</code> to continue a previous conversation.
genie_poll_response	Fetch the latest state of an in-flight or completed response, including progress steps, the final answer, and deep links.
genie_get_query_result	Fetch the full SQL result (column schema + rows) for a query Genie ran.
genie_cancel_response	Cancel an in-flight Genie turn.
view_ask	Ask a question and open the interactive View (for MCP Apps clients).

Step 4: Build Your Agent (Choose a Framework)

Option A: OpenAI Agents SDK

```

import asyncio
from agents import Agent, Runner
from databricks.sdk import WorkspaceClient
from databricks_openai.agents import McpServer

async def main():
    workspace_client = WorkspaceClient()
    host = workspace_client.config.host

    async with McpServer(
        url=f"{host}/ai-gateway/mcp-services/system.ai.genie_one_mcp",
        name="genie-one-mcp",
        workspace_client=workspace_client,
    ) as mcp_server:
        agent = Agent(
            name="Genie Data Agent",
            instructions="You are a helpful data assistant. Use Genie to answer data questions.",
            model="databricks-claude-sonnet-4-5",
            mcp_servers=[mcp_server],
        )
        result = await Runner.run(agent, "What were total sales last quarter?")
        print(result.final_output)

asyncio.run(main())

```

Option B: LangGraph

```

from databricks.sdk import WorkspaceClient
from databricks_langchain import ChatDatabricks, DatabricksMCPClient,
DatabricksMultiServerMCPClient
from langgraph.prebuilt import create_react_agent

workspace_client = WorkspaceClient()
host = workspace_client.config.host

mcp_client = DatabricksMultiServerMCPClient([
    DatabricksMCPClient(
        name="genie-one-mcp",
        url=f"{host}/ai-gateway/mcp-services/system.ai.genie_one_mcp",
        workspace_client=workspace_client,
    ),
])

async with mcp_client:
    tools = await mcp_client.get_tools()
    agent = create_react_agent(
        ChatDatabricks(endpoint="databricks-claude-sonnet-4-5"),
        tools=tools,
    )

```

```

    )
    result = await agent.ainvoke(
        {"messages": [{"role": "user", "content": "What were total sales last quarter?"}]}
    )
    print(result["messages"][-1].content)

```

Option C: Direct Tool Call (No Framework)

```

# Call genie_ask directly
result = mcp_client.call_tool("genie_ask", {
    "question": "What were total sales last quarter?"
})
print(result.content)
# Returns conversation_id, response_id, status

# Poll for the response
poll_result = mcp_client.call_tool("genie_poll_response", {
    "conversation_id": "<conversation_id>",
    "response_id": "<response_id>"
})
print(poll_result.content)

# Get full query results
query_result = mcp_client.call_tool("genie_get_query_result", {
    "conversation_id": "<conversation_id>",
    "response_id": "<response_id>"
})
print(query_result.content)

```

Step 5: Wrap It in a Databricks App

Build a web UI using Streamlit, Dash, or Gradio that calls the agent code above. Here's a simplified Streamlit example:

```

# app.py
import streamlit as st
import asyncio
from databricks.sdk import WorkspaceClient
from databricks_openai.agents import McpServer
from agents import Agent, Runner

st.title("Ask Your Data with Genie One")

question = st.text_input("Ask a data question:")

if st.button("Ask") and question:
    with st.spinner("Genie is thinking..."):
        async def ask_genie(q):
            workspace_client = WorkspaceClient()
            host = workspace_client.config.host

```

```

    async with McpServer(
        url=f"{host}/ai-gateway/mcp-services/system.ai.genie_one_mcp",
        name="genie-one-mcp",
        workspace_client=workspace_client,
    ) as mcp_server:
        agent = Agent(
            name="Genie Data Agent",
            instructions="Answer data questions using Genie.",
            model="databricks-claude-sonnet-4-5",
            mcp_servers=[mcp_server],
        )
        result = await Runner.run(agent, q)
        return result.final_output

answer = asyncio.run(ask_genie(question))
st.markdown(answer)

```

Step 6: Configure Authentication for the App

Databricks Apps support two authentication methods:

App Authorization (Default — Recommended for Shared Access)

Databricks automatically creates a service principal for your app. All users share the same permissions. Declare resources in `databricks.yml`:

```

resources:
  apps:
    genie_app:
      name: 'genie-data-app'
      source_code_path: './'
      config:
        command: ['streamlit', 'run', 'app.py']
      resources:
        - name: 'llm'
          serving_endpoint:
            name: 'databricks-claude-sonnet-4-5'
            permission: 'CAN_QUERY'

```

User Authorization (On-Behalf-of-User)

Each user's own permissions are used. This ensures Unity Catalog row/column-level security is enforced per user.

Step 7: Deploy the App

Deploy using Databricks Asset Bundles:

```

databricks bundle deploy
databricks bundle run genie_app

```

Or deploy via the workspace UI:

1. Go to **Databricks Apps** in the workspace.
2. Click **+ Create app**.
3. Upload your source code or connect a Git repo.
4. Configure compute and environment.
5. Deploy.

How the Conversation Flow Works

User types question in your App UI

↓

App calls `genie_ask("What were total sales last quarter?")`

↓

Returns: `conversation_id`, `response_id`, `status: in_progress`

↓

App polls `genie_poll_response` until `status = completed`

↓

Genie returns: answer text, SQL, visualizations, deep links, citations

↓

App renders the answer in the UI

↓

For follow-ups: reuse the same `conversation_id`

Governance and Security

- **Unity Catalog permissions** are always enforced — users can only query data they're allowed to access.
 - **Service policies** can allow or deny individual tool calls on the MCP Service.
 - **Every invocation is recorded** for usage and audit.
 - To govern who can invoke the service: go to **Unity Gateway** → **MCPs** → **system.ai.genie_one_mcp** and configure access.
-

Key References

- Genie One MCP server documentation
- Use MCP servers in Custom Agents (includes LangGraph, OpenAI, and Agents SDK example notebooks)
- Databricks Apps documentation
- Author an agent and deploy it on Databricks Apps
- Connect MCPs to AI assistants and coding agents